Search-to-decision reductions for formula minimization from near-optimal formula counts and reverse gate elimination

Rahul Ilango

doi:10.4230/LIPIcs.CCC.2020.31; CC BY3.0

Editorial note. RahulIlango. CCC2020 article31,DOI10.4230/LIPIcs.CCC.2020.31. CC BY3.0.

Editorial note. Faithful editable transcription of official CCC2020 CC BY3.0 article31,35-page version. One near-optimal-formula-count/reverse-gate-elimination search-to-decision theme: exact formal33, Cor34 and randomized41 with all supporting original Sections2–6 author proofs. Complete formula, basis, isomorphism, canonical, partial, MFSP and input N=2^n conventions10–16, decomposition17–21/Algorithm1, Select22/23 and full Lemma24 two-injection proof with every substitution/DropLeaf/AddLeaf definition and Claims25–32, complete Algorithm2/Claims35–40/correctness/recursive-count and runtime analysis, and complete Algorithm3/Claims42–52 including all sampling/candidate/partial-completion reasoning and four-part runtime/probability bounds retained. All20 original bibliography entries retained. Near-optimal count t is |CanonOpt^(n+1)Formulas(f)|, with formulas counted up to labelled-tree isomorphism. Formal33 bound O(t^2 N^6 log^2N) retained as actually printed, alongside author subroutine bounds. Cor34 applies to all but o(1) fraction of length-N truth-table instances; random41 analysis explicitly yields 99percent runtime bound, while Claim50 supplies correctness under s>=L(f)/2. Exponential randomized notation is O(2^(.67N)), not (2.67)^N. MFSP oracle query lengths N/2N, leaf-size DeMorgan basis and author stated B2/size-convention extension retained; no general polynomial-time search-to-decision solution claimed. Standard external Lozhkin/Lupanov and Pippenger statements retained without recursive prior proofs. All source proof reasoning retained, including author-stated omitted easy Claim28 justification; no added proofs. Canonical representatives displayed with an overbar and pseudocode reflowed into editable sequential/nested lists preserving every original operation; original Algorithm2 line10 is explicitly tagged. Original printed variable/index and wording inconsistencies retained without mathematical correction (comparison15 display, AddLeaf extraD in final injection, i_p/v_p, source equality/inequality and part4 runtime lines). Section7 bottom-up alternative is outside this selected top-down theme, not counted separately. No required figures. Root handles compilation/database/publication.

Allowed standard prerequisites (editorial).

- Maximum formula-size upper bound. Original Theorem13 Lozhkin[11]/Lupanov[12] retained; current-author near-optimal-count Prop14 full proof included.

- Partial-function formula-size bound. Original Theorem16 Pippenger[16] retained; full current-author Claim52 deduction included.
- Uniform random binary tree generation. Original Section6.2 linear-time statement and[15] citation retained.

Theorem (Author formal Theorem33)**.** *There is a deterministic algorithm solving Search-MFSP on inputs of length N given access to an oracle that solves MFSP on instances of length $2N$ that runs in time $O(|\operatorname{CanonOpt}^{n+1}\operatorname{Formulas}(f)|^2 \cdot N^6 \log^2 N)$ where $n = \log N$.*

Theorem (Author Corollary34)**.** *There is a deterministic algorithm solving Search-MFSP on inputs of length N given access to an oracle that solves MFSP on instances of length $2N$ that runs in time $2^{O(N/\log\log N)}$ on all but a $o(1)$-fraction of instances.*

Theorem (Author Theorem41)**.** *There is a randomized algorithm solving Search-MFSP on inputs of length N in time $O(2^{.67N})$ given access to an oracle that solves MFSP on instances of length $2N$.*

2 Preliminaries

For a positive integer n, we let $[n]$ denote the subset of integers $\{1, \ldots, n\}$.

2.1 DeMorgan Formulas and Formula Size

Our notion of formulas will be DeMorgan formulas. A DeMorgan formula ϕ on n-inputs of size s is given by:

- a directed rooted binary tree on the vertex set $[2s-1]$, specified by a subset $E_\phi \subseteq [2s-1] \times [2s-1]$ of edges, and
- a gate labeling function $\tau_\phi : [2s-1] \to \{\wedge, \vee\} \cup \{0, 1, x_1, \ldots, x_n, \neg x_1, \ldots, \neg x_n\}$

where τ takes values in $\{\wedge, \vee\}$ on the internal nodes in ϕ and τ takes values in $\{0, 1, x_1, \ldots, x_n, \neg x_1, \ldots, \neg x_n\}$ on the leaf nodes in ϕ. The edges in E_ϕ point from inputs towards outputs. We note that our definition implicitly uses the fact that a binary tree with s leaf nodes has $s-1$ internal nodes. We also note that in our definition we do not need to specify the "left" and "right" child of an internal node since our gate set $\{\wedge, \vee\}$ is made up of symmetric functions. We will define a notion of formula isomorphism in Section 2.2. We will use the notation $|\phi|$ to denote the size of a formula ϕ (i.e. the number of leaves in the binary tree underlying ϕ). Given a Boolean function f, we denote the minimum formula size of f by

$$L(f) = \min\{|\phi| : \phi \text{ is a formula computing } f\}.$$

We say a formula ϕ is an optimal formula for a Boolean function f, if ϕ computes f and $|\phi| = L(f)$. We note, however, that all of our results except the ones presented in Section 7 apply equally well to formulas with arbitrary fan-in-two gates (i.e. the formulas over the B_2 basis). Moreover, all our results are hold for other size notions such as gates and wires.

2.2 Optimal Formulas and Formula Isomorphism

Since our results will depend on the number of formulas satisfying certain properties, we will be clear about when exactly we are saying formulas are distinct in our count. In particular, as we have defined formulas, one can obtain many optimal formulas from a single optimal formula by relabeling the nodes in underlying binary tree. Thus, it will be useful to define an isomorphism on formulas and only count formulas modulo this isomorphism. In particular, we will define two formulas to be isomorphic if they are isomorphic as labelled binary trees. In order to properly define this, we introduce some notation. If ϕ is a formula of size s with an underlying edge set E_ϕ and a labelling function τ_ϕ and $\sigma : [2s-1] \to [2s-1]$ is a permutation, then we let $\psi = \sigma(\phi)$ be the formula of size s whose edge set E_ψ is given by

$$E_\psi = \{(\sigma(i), \sigma(j)) : (i,j) \in E_\phi\}$$

and whose labelling function τ_ψ is given by $\tau_\psi(\sigma(i)) = \tau_\phi(i)$. We say two formulas ϕ and ϕ' are isomorphic if $|\phi| = |\phi'|$ and there is a permutation σ such that $\phi' = \sigma(\phi)$. From each equivalence class of isomorphic formulas, we pick a single representative that we call the canonical formula for that equivalence class. Note that for our purposes we do not need that this canonical formula to be computable, as we will just be using them in our analysis. Then we define $\text{CanonOpt}^k\text{Formulas}(f)$ to be the set of canonical formulas that are optimal for computing f up to an additive k-term. In other words

$$\text{CanonOpt}^k\text{Formulas}(f) = \{\phi : \phi \text{ is a canonical formula and } |\phi| \leq L(f) + k\}.$$

2.3 MFSP, Search-MFSP and Conventions on n and N

We now define the Minimum Formula Size Problem denoted MFSP.

Definition 10 (Author Definition 10: MFSP)**.** *We define the problem MFSP as follows*

Given: *a truth table of a Boolean function f and an integer size parameter $s \geq 1$*

Determine: *if $L(f) \leq s$.*

We define the search version of MFSP analogously.

Definition 11 (Author Definition 11: Search-MFSP)**.** *Search-MFSP is the problem defined as follows:*

Given: *a truth table of a Boolean function f*

Output: *a formula ϕ of size $L(f)$ computing f.*

We note that MFSP$\in$ NP since given a minimum-sized formula as a witness, one can check that this indeed computes f efficiently since the truth table of f is provided and every function has a formula of size at most the length of its truth table (see Theorem 13). When describing a function f that is an input to MFSP, one naturally wants to denote by n two different quantities: the number of variable inputs to a function f and the length of the truth table of f (which is the true input length for MFSP). We maintain the convention throughout this paper that n denotes the input arity of f and $N = 2^n$ denotes the length of the truth table of f.

2.4 Useful Facts About Formulas

We will make use of some basic facts about formulas in our work. First, one can easily bound the number of formulas of size at most s.

Proposition 12 (Author Proposition 12)**.** *The number of formulas on n-inputs of size at most s is at most $2^{s \log n(1+o(1))}$.*

We also know tight upper bounds on the maximum formula complexity of a n-input function.

Theorem 13 (Author Theorem 13: Lozhkin [11] improving on Lupanov [12])**.** *Let $f : \{0,1\}^n \to \{0,1\}$. Then*

$$L(f) \leq \frac{2^n}{\log n}\left(1 + O\left(\frac{1}{\log n}\right)\right).$$

Combining the size upper bound in Theorem 13 with the bound on the number of formulas of size s, we get the following proposition.

Proposition 14 (Author Proposition 14: Random functions have not too many near optimal formulas)**.** *Let n and k be positive integers. Let $N = 2^n$. Assume $k = O(2^n/\log^2 n)$. Then all but a $o(1)$-fraction of n-input Boolean functions f satisfy*

$$|\operatorname{CanonOpt}^k\operatorname{Formulas}(f)| = 2^{O(N/\log\log N)}.$$

Proof. Theorem 13 say that every n-input function has a formula of size at most $\frac{2^n}{\log n}(1 + O(\frac{1}{\log n}))$. Thus, any formula for computing n-input function that is within an additive k of being optimal has size at most s where

$$s \leq k + \frac{2^n}{\log n}(1 + O(\tfrac{1}{\log n})) = \frac{2^n}{\log n}(1 + O(\tfrac{1}{\log n})).$$

Proposition 12 implies that the number of formulas of size at most s is upper bounded by

$$2^{s \log n(1+o(1))} = 2^{N(1+O(1/\log\log N))}.$$

Hence, since there are 2^N Boolean functions on n-inputs, it follows that in expectation a random function has at most $2^{O(N/\log\log N)}$ formulas within k of being optimal. The desired claim then follows by an application of Markov's inequality. □

We note that the bound given by Proposition 14 is actually counting formulas that are isomorphic to each other as distinct. Unfortunately removing this redundancy does not improve on the bound in Proposition 14. However, the fact that our results rely on the number of distinct formulas up to isomorphism means that there is no obvious obstruction to better bounds being proved and hence to our algorithms being more efficient. We will also make use of the fact that integer comparison can be implemented by linear-sized formulas.

Proposition 15 (Author Proposition 15: Small formulas for integer comparison)**.** *Let $y \in \{0,1\}^n$. Let $\operatorname{GrtrThan}_y : \{0,1\}^n \to \{0,1\}$ be the function given by $\operatorname{GrtrThan}_y(x) = 1$ if and only if $x > y$ in the usual lexicographic order on $\{0,1\}^n$. Then $L(\operatorname{GrtrThan}_y(x)) \leq n$.*

Proof. We work by induction on n. If $n = 1$, then clearly $L(\text{GrtrThan}_y) = 1$ (either it is 0 if $y = 1$ or it equals x if $y = 0$). Now suppose $n > 1$. Let $x_1, \ldots, x_n$ and $y_1, \ldots, y_n$ denote the bits of x and y respectively where x_1 and y_1 denotes the highest order bit. Let $x', y' \in \{0,1\}^{n-1}$ be given by $x' = x_2 \ldots x_n$ and $y' = y_2 \ldots y_n$ respectively. Now, $x > y$ if and only if one of the following two statements is true:

- $x_1 > y_1$, or
- $x_1 = y_1$ and $x' > y'$.

Since $x_1, y_1 \in \{0,1\}$, this is equivalent to

$$x > y \quad \iff \quad (x_1 > y_1) \vee (x' > y').$$

By induction this means $L(\text{GrtrThan}_y) \leq 1 + n - 1 = n$. □

2.5 Partial Functions and their Formula Size

Partial functions will be a crucial building block in our reductions. A partial Boolean function is a function $\gamma : \{0,1\}^n \to \{0,1,?\}$ for some integer $n \geq 1$. We denote partial functions using Greek letters such as γ and μ, although sometimes we resort to the Roman alphabet with a? subscript such as $h_?$. In contrast, we say a Boolean function $f : \{0,1\} \to \{0,1\}$ is total Boolean function (though we allow for a partial Boolean function to indeed be total). We say a total Boolean function g agrees with a partial Boolean function γ if

$$\gamma(x) \in \{0,1\} \quad \implies \quad \gamma(x) = g(x).$$

One can naturally define the minimum formula size of a partial Boolean function γ as follows

$$L(\gamma) = \min\{L(g) : g \text{ is a total function that agrees with } \gamma\}.$$

The following theorem regarding the formula complexity of partial functions will be useful in our randomized worst-case reduction.

Theorem 16 (Author Theorem 16: Pippenger [16])**.** *Let $\gamma : \{0,1\}^n \to \{0,1,?\}$ be a partial function. Let $p_? = |\gamma^{-1}(?)|/2^n$. Then,*

$$L(\gamma) \leq (1 + o(1)) \cdot (1 - p_?) \frac{2^n}{\log n}.$$

3 The Top-Down Approach

Our two main reductions both take a "top-down" approach to finding an optimal formula. That is, given a function f, they try to find functions g and h such that g and h are the two functions fed into the final output gate in an optimal formula for f and then recursing. This is formalized as follows.

Definition 17 (Author Definition 17: Optimal Subcomputations Set)**.** *Let $f : \{0,1\}^n \to \{0,1\}$. We define the set of optimal subcomputations for f, denoted* $\text{OptSubcomps}(f)$*, as follows. Let $g, h : \{0,1\}^n \to \{0,1\}$ be Boolean functions of the same arity as f and $O \in \{\wedge, \vee\}$. Then $\{g, O, h\} \in \text{OptSubcomps}(f)$ if and only if there exists an optimal formula $\phi = \phi_g O \phi_h$ for computing f such that ϕ_g computes g and ϕ_h computes h.*

We note that in this definition we are implicitly using that the gate set $\{\wedge, \vee\}$ is symmetric with respect to its inputs. We say a function g is in an optimal subcomputation for f if g is contained in some element of OptSubcomps(f). In other words, g is in an optimal subcomputation for f if there exists an h and O such that $\{g, O, h\} \in$ OptSubcomps(f). It is easy to see that OptSubcomps(f) is almost always non-empty.

Proposition 18 (Author Proposition 18)**.** *Let $f : \{0,1\}^n \to \{0,1\}$ such that $L(f) \geq 2$. Then* OptSubcomps(f) *is non-empty.*

Next, we can define the problem of finding an optimal subcomputation.

Definition 19 (Author Definition 19: Decomposition Problem)**.** *The Decomposition Problem, DecompProblem is as follows:*

Given: *the truth table of a Boolean function f satisfying $L(f) \geq 2$*

Output: *some element of* OptSubcomps(f).

It is easy to see that DecompProblem is equivalent to Search-MFSP. DecompProblem can be easily solved with an oracle to Search-MFSP. The following recursive procedure shows the reverse direction.

Theorem 20 (Author Theorem 20: Search-MFSP reduces to DecompProblem)**.** *There is a deterministic $O(N^2)$-time algorithm for solving Search-MFSP on inputs of length N given access to an oracle that solve DecompProblem on instances of length N.*

Proof. The pseudocode for this reduction is written in Algorithm 1.

Algorithm 1. Reduction from Search-MFSP to DecompProblem.

procedure FindOptFormula(f)

Given the length-N truth table of a function f that takes n-inputs and oracle access to DecompProblem return an optimal formula for f.

1. If there exists a size one formula ϕ computing f, then return ϕ. End if.
2. Let $\{g, O, h\}$ be the output returned by the oracle DecompProblem(f).
3. Recursively compute the formula $\phi_g \leftarrow$ FindOptFormula(g).
4. Recursively compute the formula $\phi_h \leftarrow$ FindOptFormula(h).
5. Return the formula given by $\phi_g O \phi_h$.

end procedure

The correctness of this algorithm is easy to see as long as one is able to bound the number of recursive calls the algorithm makes. To see that the number of recursive calls is bounded by $O(N)$, notice that each iteration of the algorithm reveals one more gate in the optimal formula for f. Thus, since $L(f) = O(N)$, we have that there are at most $O(N)$ recursive calls. □

Our goal is now to try to solve DecompProblem (i.e. find an element of OptSubcomps(f)) given an oracle to MFSP. Recall from the introduction that our high-level approach is as follows

1. Find an efficient "test" that functions that in an optimal subcomputation of f pass but not too many other functions pass.
2. Efficiently build the (not too long) list Candidates of things that pass the test.
3. Iterate through all pairs of elements in Candidates and all possible gates, and efficiently check if this yields an element of OptSubcomps(f).

Item 1 will be the subject of Section 4, Item 2 will be different in our two main reductions, and Item 3 is provided by the next lemma.

Lemma 21 (Author Lemma 21: Test membership in OptSubcomps(f) efficiently with MFSP)**.** *Let* $f, g, h : \{0,1\}^n \to \{0,1\}$*. Then*

$$\{g, O, h\} \in \text{OptSubcomps}(f) \quad \iff \quad f = gOh \text{ and } L(f) = L(g) + L(h).$$

Proof. We prove the forward direction first. Suppose that $\{g, O, h\} \in \text{OptSubcomps}(f)$. Then there exists an optimal formula $\phi = \phi_g O \phi_h$ for computing f such that ϕ_g computes g and ϕ_h computes h. Clearly this implies that $f = gOh$. Moreover, $|\phi| = |\phi_g| + |\phi_h|$. On the other hand, since ϕ is optimal, we have that $|\phi| = L(f)$, $|\phi_g| = L(g)$, and $|\phi_h| = L(h)$. (Otherwise, one could build a smaller formula for f by replacing ϕ_g or ϕ_h with a smaller formula computing the same function.) Hence $L(f) = L(g) + L(h)$. For the reverse direction, suppose that $L(f) = L(g) + L(h)$ and $f = gOh$. Let ϕ_g and ϕ_h be optimal formulas for g and h. Then $\phi = \phi_g O \phi_h$ clearly computes f and has size $L(f)$. Hence $\{g, O, h\} \in \text{OptSubcomps}(f)$. □

4 Using gate elimination to find functions in an optimal subcomputation

Our approach to solving DecompProblem involves finding a "test" that functions in an optimal subcomputation pass but not too many other functions pass. The test will be based off the following function.

Definition 22 (Author Definition 22: Select$[\cdot,\cdot]$)**.** *Let* $f, g : \{0,1\}^n \to \{0,1\}$*. We define the function* $\text{Select}[f,g] : \{0,1\}^n \times \{0,1\} \to \{0,1\}$ *by*

$$\text{Select}[f,g](x,z) = \begin{cases} f(x), & \text{if } z = 0, \\ g(x), & \text{if } z = 1. \end{cases}$$

We emphasize that Select$[f,g]$ function is only defined when f and g have the same arity. Now, our "test" will be to see if the quantity $L(\text{Select}[f,g]) - L(f)$ is small (how small will depend on our reduction). Indeed, for functions in an optimal subcomputation, this quantity is exactly one![1]

[1]We will only prove that it is as most one, but the reader can check that if $g \neq f$ that a gate elimination argument actually implies equality.

Lemma 23 (Author Lemma 23)**.** *Suppose g is in an optimal subcomputation for f. Then $L(\text{Select}[f,g]) \leq L(f)+1$.*

Proof. Since g is in an optimal subcomputation for f, there exists an optimal formula $\phi = \phi_g O \phi_h$ such that ϕ_g computes g. If $O = \wedge$, then $\phi_g \wedge (\phi_h \vee z)$ is a formula for $\text{Select}[f,g]$ of size $L(f)+1$. Otherwise $O = \vee$. Then $\phi_g \vee (\phi_h \wedge \neg z)$ is a formula for $\text{Select}[f,g]$ of size $L(f)+1$. □

On the other hand, the number of functions that "pass this test" can be upper bounded in terms of $|\,\text{CanonOpt}^k\text{Formulas}(f)|$.

Lemma 24 (Author Lemma 24)**.** *Let k be a positive integer. Let $f : \{0,1\}^n \to \{0,1\}$. Assume $L(f) \geq 2$. Let* $\text{TestPassers} = \{g : L(\text{Select}[f,g]) - L(f) \leq k+1\}$. *Then*

$$|\,\text{TestPassers}\,| \leq O(|\,\text{CanonOpt}^k\text{Formulas}(f)|\cdot 2^k N \log N)$$

where $N = 2^n$.

Proof. At a high-level the idea is that, given a formula ϕ of size $L(f)+k+1$ for computing $\text{Select}[f,g]$, one can replace the z-leaves in ϕ with 0-leaves to obtain a formula ϕ' of size $L(f)+k+1$ for computing f with at least one constant leaf. One can then use a careful gate elimination argument to remove precisely one constant leaf from ϕ' to obtain a formula ϕ'' that still computes f but has size $L(f)+k$. On the other hand, one can reverse this process by adding some constant leaf and gate to ϕ'' and then replacing some subset of the constant leaves by z-leaves. This gives us a way to describe any g that passes the test, and thus allows us to bound the number of such g. In our bound, the $O(|\,\text{CanonOpt}^k\text{Formulas}(f)|)$ factor corresponds to the choices for ϕ'', the $O(N \log N)$ corresponds to the number of ways to add a new constant leaf and gate to ϕ'' in order to obtain ϕ', and the $O(2^k)$ factor comes from the number of ways to chose a subset of the (at most k)0-leaves in ϕ' into z-leaves. In detail, we prove this statement by giving a series of injections. At a high-level First, let P denote the set of canonical formulas computing f with size exactly $L(f)+k+1$ and with at least one constant-labelled leaf node in the formula. We will give an injection from TestPassers to $P \times [2^{k+1}]$. Before defining our injection, we will need the following claims and definitions.

Claim 25 (Author Claim 25)**.** *Suppose ϕ computes* $\text{Select}[f,g]$ *and $f \neq g$, then ϕ has at least one leaf node labelled by z or $\neg z$.*

Proof. If ϕ does not have any $\{z, \neg z\}$ labelled leaves, then the output of ϕ does not depend on z. But this contradicts that ϕ computes $\text{Select}[f,g]$ since $\text{Select}[f,g]$ does depend on the z input because $f \neq g$. □

Claim 26 (Author Claim 26)**.** *Suppose $\phi \in P$. Then ϕ has at most $(k+1)$-many leaf nodes labelled by constants $\{0,1\}$.*

Proof. Since $\phi \in P$, we know that $|\phi| = L(f)+k+1$ and ϕ computes f. Since $L(f) \geq 2$, we know that $|\phi| \geq k+3$. If ϕ had more than $(k+1)$-many constant labelled leaves, it follows by a standard gate elimination argument (note here it is important that $|\phi| \geq k+3$) that there is a ϕ' that computes the same function as ϕ such that

$$|\phi'| < |\phi| - (k+1) < L(f).$$

But then ϕ would be a formula of size less than $L(f)$ computing f which is a contradiction. $\square$

We will also need the following definitions. Given a formula ϕ that can take z-variables as input, we define $\text{Substitute}_{z=0}(\phi)$ to be the formula ϕ' given by replacing the z-labeled leaves in ϕ with 0-labels and replacing the $(\neg z)$-labeled leaves in ϕ with 1-labels. We note that the $\text{Substitute}_{z=0}$ operation in some sense respects formula isomorphisms.

Claim 27 (Author Claim 27)**.** *Let ϕ be a formula of size s that takes a z-variable as input. Let $\sigma : [2s-1] \to [2s-1]$ be a permutation. Then*

$$\sigma \circ \text{Substitute}_{z=0}(\phi) = \text{Substitute}_{z=0} \circ \sigma(\phi).$$

Proof. The proof is essentially just applying the definition to both sides and seeing that the resulting edge sets and labelling functions are equal. $\square$

We can also define a reverse operation to $\text{Substitute}_{z=0}$ as follows. Given a formula ϕ and a subset S of leaf nodes in ϕ that are labelled by constants $\{0,1\}$, define $\text{Unsubstitute}_{z=0}(\phi', S)$ to be the formula ϕ given by replacing 0-labeled leaves in S with z-labels leaves and by replacing 1-labeled leaves in S with $(\neg z)$-labels. Indeed, the following claim whose proof we omit is easy to see.

Claim 28 (Author Claim 28)**.** *For all formulas ϕ there exists a set S such that*

$$\phi = \text{Unsubstitute}_{z=0}(\text{Substitute}_{z=0}(\phi), S).$$

Being more precise, S is a subset of the leaf nodes in $\text{Substitute}_{z=0}(\phi)$ that are labelled by constants. Now we are ready to describe our injection from $\text{TestPassers} \to P \times [2^{k+1}]$ on an input $g \in \text{TestPassers}$. Since $g \in \text{TestPassers}$, there is a ϕ of size $L(f)+k+1$ computing $\text{Select}[f,g]$. Let $\phi' = \text{Substitute}_{z=0}(\phi)$. Clearly ϕ' computes f since ϕ computes $\text{Select}[f,g]$. Let $\overline{\phi'}$ denote the canonical formula isomorphic to ϕ'. Then there exists a permutation σ such that

$$\begin{aligned}\overline{\phi'} &= \sigma(\phi') \\ &= \sigma \circ \text{Substitute}_{z=0}(\phi) \\ &= \text{Substitute}_{z=0} \circ \sigma(\phi)\end{aligned}$$

where the last equality comes from Claim 27. Thus, using Claim 28, we know that there exists a subset S of the leaf nodes of $\overline{\phi'}$ labelled by constants such that

$$\text{Unsubstitute}_{z=0}(\overline{\phi'}, S) = \sigma(\phi).$$

Moreover, the set S can be viewed as an element of $[2^{k+1}]$ because $\overline{\phi'}$ has at most $k+1$ leaf nodes labelled by constants. In particular, by construction, we have that

$$|\overline{\phi'}| = |\phi'| = |\phi| = L(f) + k + 1,$$

and that $\overline{\phi'}$ computes f, so Claim 26 ensures that $\overline{\phi'}$ has at most $k+1$ many leaf nodes labelled by constants. Hence, we define the output of our injection from TestPassers to $P \times [2^{k+1}]$ on input $g \in \text{TestPassers}$ to be $(\overline{\phi'}, S)$. We must prove that this is indeed an injection. Towards this end, we claim that $\text{Unsubstitute}_{z=0}(\overline{\phi'}, S)$ is a formula computing $\text{Select}[f,g]$. From this claim it is easy to see that this must be an injection.

Claim 29 (Author Claim 29)**.** $\text{Unsubstitute}_{z=0}(\overline{\phi'}, S)$ *is a formula computing* $\text{Select}[f, g]$.

Proof. S was chosen so that $\text{Unsubstitute}_{z=0}(\overline{\phi'}, S) = \sigma(\phi)$. Thus, $\text{Unsubstitute}_{z=0}(\overline{\phi'}, S)$ computes the same function as $\sigma(\phi)$ which in turn computes the same function as ϕ, which computes $\text{Select}[f, g]$ as desired. □

From this injection, we get that $|\,\text{TestPassers}\,| \leq 2^{k+1} \cdot |P|$.
Next, we give an injection from P to the set

$$\text{CanonOpt}^k\text{Formulas}(f) \times [2L(f)] \times \{\wedge, \vee\} \times \{0, 1, x_1, \ldots, x_n, \neg x_1, \ldots, \neg x_n\}.$$

To do this we will define an operation $\text{DropLeaf}(\phi, i)$ that takes as input a formula ϕ of size $s \geq 2$ and a leaf node $i \in [2s-1]$ from ϕ and outputs the formula ϕ' given as follows. We will first describe ϕ' informally and then give the formal description. ϕ' is obtained by deleting the leaf node i and making the output of the node i_p that i fed into simply the other node that was being fed into i_p. Now, we formally describe ϕ'. Let $i_p \in [2s-1]$ be the internal node that i has an edge to in ϕ (we know this exists because $|\phi| \geq 2$). If needed, apply a permutation to ϕ so $i = 2s-1$ and $i_p = 2s-2$. Let $u \in [2s-3]$ be the other node in ϕ that feeds into v_p. Let ϕ' be the formula given by the edge set

$$E' = (E \cap ([2s-3] \times [2s-3])) \cup \{(u, v_{pp}) : v_p \text{ feeds into } v_{pp} \text{ in } \phi\}$$

and the labelling function $\tau' = \tau|_{[2s-3]}$. For example if $\phi = (x_1 \vee x_2) \wedge x_3$ and 1 was then index of the x_1 leaf, then $\text{DropLeaf}(\phi, 1) = x_2 \wedge x_3$. We show this operation in some sense commutes with formula isomorphisms.

Claim 30 (Author Claim 30)**.** *Let ϕ be formula of size s. Let i be the index a leaf node in ϕ, and let $\sigma : [2s-3] \to [2s-3]$ be a permutation. Then there exists an integer i' and a permutation $\sigma' : [2s-1] \to [2s-1]$ such that*

$$\text{DropLeaf}(\sigma'(\phi), i') = \sigma \circ \text{DropLeaf}(\phi, i).$$

Proof. From our definition of $\text{DropLeaf}(\cdot, \cdot)$, we can assume without loss of generality that $i = 2s-1$ and that the internal node that i feeds into in ϕ is $i_p = 2s-2$. Then the claim follows from letting σ be equal to σ' on $[2s-3]$ and letting σ be the identity on $\{2s-2, 2s-1\}$ and applying the various definitions. □

We can also define a kind of inverse operation AddLeaf function that takes the following four inputs

- a formula ϕ' on n-inputs of size s,
- a node i in the tree given by ϕ',
- a gate $O \in \{\wedge, \vee\}$, and
- a leaf label $\ell \in \{0, 1, x_1, \ldots, x_n, \neg x_1, \ldots, \neg x_n\}$,

and outputs the formula ϕ of size $s+1$ given as follows. First, we give an informal description and then given a formal definition. Intuitively, AddLeaf is adding a new O-gate into ϕ' between the i-node and wherever i was being output to (if i has an output), whose other input is a new ℓ-labeled leaf. We define ϕ formally as follows. We will use $2s+1$ to add in our new leaf and $2s$ to add in our new gate. The edge set E_ϕ of ϕ is given by taking $E_{\phi'}$ and adding in the edges $(2s+1, 2s)$ and $(i, 2s)$ and then, if there is a node i_p that i feeds into in ϕ', adding in an edge $(2s, i_p)$ and removing the (i, i_p) edge. The node labelling τ_ϕ of ϕ is given by

$$\tau_\phi(i) = \begin{cases} \tau_{\phi'}, & \text{if } i \in [2s-1], \\ O, & \text{if } i = 2s, \\ \ell, & \text{if } i = 2s+1. \end{cases}$$

It is easy to see that AddLeaf can reverse a DropLeaf$(\cdot,\cdot)$ operation.

Claim 31 (Author Claim 31)**.** *Let ϕ be a formula of size at least two. Let i be the index of a leaf node in ϕ. Then there exists an integer j, a gate $O \in \{\wedge, \vee\}$, and a leaf label $\ell \in \{0, 1, x_1, \ldots, x_n, \neg x_1, \ldots, \neg x_n\}$ such that*

$$\text{AddLeaf}(\text{DropLeaf}(\phi, i), j, O, \ell) = \phi.$$

One of the main steps in our injection will be provided by the following claim.

Claim 32 (Author Claim 32)**.** *Let $\phi \in P$. Then there is an i such that $\phi' = \text{DropLeaf}(\phi, i)$ is a size $(L(f)+k)$ formula for computing f.*

Proof. Let $\phi \in P$. Then there is some internal node j in ϕ that takes as input a leaf node indexed by i satisfying $\tau_\phi(i) = b \in \{0, 1\}$. We will assume $b = 0$ (the proof in the $b = 1$ case is similar). Let ϕ_j be the the subformula computed at node j, and let $O_j = \tau_\phi(j) \in \{\wedge, \vee\}$ be the gate label of j. We already know that the ith leaf node feeds into ϕ_j. Let k be the other node feeding into j, and let ϕ_k be the subformula computed at node k. We split into cases depending on whether O_j is an $\wedge$-gate or a $\vee$-gate. First, let us suppose O_j is a $\vee$-gate. Then the formula ϕ_j as a function is equivalent to the formula $0 \vee \phi_k$ which is equivalent as a function to ϕ_k. Hence it follows that $\phi' = \text{DropLeaf}(\phi, i)$ is an $(L(f)+k)$-size formula (since we removed the ith leaf) computing f (since ϕ_j and ϕ_k compute the same function). Now, suppose that it is a $\wedge$-gate. Then the output of ϕ_j is always zero. Since $|\phi_j| \geq 2$, there exists some subformula ϕ_2 of ϕ_j of size 2 (i.e. there is some node in ϕ_v that has two leaves as children and ϕ_2 is the subformula computed at that node). Since ϕ_2 has two leaves, there exists one leaf index i' such that $i' \neq i$ (i.e. this leaf node is not the ith leaf node we were considering before). Then we claim that $\phi' = \text{DropLeaf}(\phi, i')$ is an $(L(f)+k)$-size formula computing f. It is easy to see that $|\phi'|$ is an $(L(f)+k)$-size formula since we removed the i'th leaf node. To see that ϕ' still computes f, note that the 0-labeled ith leaf node in ϕ still exists in ϕ'. If the gate node j was removed by the DropLeaf$(\cdot,\cdot)$ operation, then the output wire of ϕ_j that computed the 0 function in ϕ has been replaced by the 0-leaf i in ϕ' which still computes the 0 function, so ϕ' must still compute f. If the gate node v was not removed by the DropLeaf$(\cdot,\cdot)$ operation, then the output corresponding gate

to v in ϕ' is still computing 0 (since it is an $\wedge$-gate with a 0 input), so ϕ' still computes f. □

Now we can finally describe the injection from P to the set

$$\text{CanonOpt}^k\text{Formulas}(f)\times[2L(f)+2k-1]\times\{\wedge,\vee\}\times\{0,1,x_1,\ldots,x_n,\neg x_1,\ldots,\neg x_n\}.$$

Given an input ϕ in P, we have by Claim 32 that there exists a least i-value such that $\phi' = \text{DropLeaf}(\phi, i)$ is a formula computing f of size $L(f) + k$. Let $\overline{\phi'}$ be the canonical formula isomorphic to ϕ'. Then we have that $\overline{\phi'} \in \text{CanonOpt}^k\text{Formulas}(f)$ and we have that there are permutations σ and σ' and an integer i' such that

$$\begin{aligned}\overline{\phi'} &= \sigma(\phi') \\ &= \sigma \circ \text{DropLeaf}(\phi, i) \\ &= \text{DropLeaf}(\sigma'(\phi), i')\end{aligned}$$

where the last equality comes from Claim 30. Hence, by Claim 31, we have that there exists a gate index $j \in [2L(f) + 2k - 1]$, a gate $O \in \{\wedge, \vee\}$, and a leaf label $\ell \in \{0, 1, x_1, \ldots, x_n, \neg x_1, \ldots, \neg x_n\}$ such that

$$\text{AddLeaf}(\overline{\phi'}, j, O, \ell, D) = \sigma'(\phi).$$

In other words, $\text{AddLeaf}(\overline{\phi'}, j, O, \ell, D)$ outputs a formula isomorphic to ϕ. Thus, we set the output of the injection on input ϕ to be $(\overline{\phi'}, j, O, \ell, D)$. The fact that this is an injection from P is ensured by the fact that $\text{AddLeaf}(\overline{\phi'}, j, O, \ell, D)$ is isomorphic to ϕ and the fact that P contains only canonical formulas. Hence, we get that

$$\begin{aligned}|P| &\leq |\,\text{CanonOpt}^k\text{Formulas}(f)| \cdot 2(L(f) + k) \cdot 2 \cdot (2n + 1) \\ &\leq O(|\,\text{CanonOpt}^k\text{Formulas}(f)|(N + k)\log N).\end{aligned}$$

Combining this with upper bound on TestPassers in terms of P, we get that

$$|\,\text{TestPassers}\,| \leq O(|\,\text{CanonOpt}^k\text{Formulas}(f)| \cdot 2^k N \log N).$$

□

5 A deterministic reduction that works on average

We will now use the tools developed in Section 3 and Section 4 to give a search to decision reduction that is efficient on functions with few near-optimal formulas.

Theorem 33 (Author Theorem 33)**.** *There is a deterministic algorithm solving Search-MFSP on inputs of length N given access to an oracle that solves MFSP on instances of length $2N$ that runs in time*

$$O(|\,\text{CanonOpt}^{n+1}\text{Formulas}(f)|^2 \cdot N^6 \log^2 N)$$

where $n = \log N$.

Before we prove Theorem 33, we state a corollary that follows from the bound on the size of CanonOpt^{k}Formulas(f) for a random function given in Proposition 14.

Corollary 34 (Author Corollary 34)**.** *There is a deterministic algorithm solving Search-MFSP on inputs of length N given access to an oracle that solves MFSP on instances of length $2N$ that runs in time $2^{O(N/\log\log N)}$ on all but a $o(1)$-fraction of instances.*

Author proof of Corollary 34. This corollary follows by combining the algorithm in Theorem 33 with the recursive algorithm for Search-MFSP given in Theorem 20 and using Proposition 14 to bound $|\text{CanonOpt}^{n+1}\text{Formulas}(f)|$ by $2^{O(N/\log\log N)}$ for a random function. There is actually a subtlety in appealing to Theorem 20 in that the running time of the algorithm in Theorem 33 has a dependence on the number near-optimal formulas of its input. Hence, we need to argue that when the algorithm in Theorem 20 makes recursive calls to the algorithm in Theorem 20 on functions g other than the original input f that g also has few near-optimal formulas. However, this is not a problem since it is easy to see that if a function g is computed by some gate in an optimal formula for f (as it must be if a recursive call is made to g), then the number of near-optimal formulas for g is at most the number of near-optimal formulas for f (since one can create a near optimal formula for f by taking the optimal formula for f that computes g at some gate and replacing the subformula at that gate with a near-optimal formula for g). □

Author proof of Theorem 33. We provide the pseudocode of our DecompProblem algorithm in Algorithm 2, which we recommend the reader look at before proceeding.

Algorithm 2. A deterministic search to decision reduction for MFSP whose run time depends on the number of "near-optimal formulas".

procedure OptimalSubcomputation(f)

Given the length-N truth table of a function f that takes n-inputs with $L(f) \geq 2$, this procedure returns an element $\{g, O, h\}$ of OptSubcomps(f).

Part 1: Building a Candidates list Let allUnknown : $\{0,1\}^n \to \{0,1,?\}$ be given by allUnknown(x) =? for all x. Set PartialCandidates$^{(0)}$ = {allUnknown}. Set $i = 0$. **while** $i < N$ **do**

1. Set PartialCandidates$^{(i+1)} = \varnothing$.
2. For all $\gamma \in$ PartialCandidates$^{(i)}$ and for all $b \in \{0,1\}$ do
   (a) Let $x^?$ (original line 10) be the lexicographically first input satisfying $\gamma(x^?) =?$.
   (b) Let $\gamma_b : \{0,1\}^n \to \{0,1,?\}$ be given by $\gamma_b(x) = \begin{cases} b, & \text{if } x = x^?, \\ \gamma(x), & \text{otherwise.} \end{cases}$
   (c) Let g_{γ_b} be the (total) function given by $g_{\gamma_b}(x) = \begin{cases} 1, & \text{if } \gamma_b(x) =?, \\ \gamma_b(x), & \text{otherwise.} \end{cases}$

(d) If $L(\text{Select}[f, g_{\gamma_b}]) \leq L(f)+n+2$ then add γ_b to PartialCandidates$^{(i+1)}$. End if.

End for.

3. Set $i = i + 1$.

end while. Set Candidates = PartialCandidates$^{(N)}$.

Part 2: Finding an optimal pair within Candidates For all pairs $g, h \in$ Candidates and for all gates $O \in \{\wedge, \vee\}$ do: If $L(g)+L(h) = L(f)$ and $f = gOh$ then return $\{g, O, h\}$. End if. End for.

end procedure

5.1 Correctness of Algorithm 2

In this subsection we show that Algorithm 2 has the desired input/output behavior. Fix some function f with n-inputs satisfying $L(f) \geq 2$. Let $N = 2^n$.

Part 1: building Candidates First, we will prove some loop invariants that will help us show that Candidates and PartialCandidates$^{(i)}$ contain those functions we are interested in and do not contain many more things. The following claim shows that the $x^?$ described on Line 10 always exists and that the ?-values of partial functions in PartialCandidates$^{(i)}$ always have an easily computable structure.

Claim 35 (Author Claim 35)**.** *Before and after each iteration of the while loop, it is true that if* $\gamma \in$ PartialCandidates$^{(i)}$*, then*

$$\gamma(x) =? \quad \iff \quad x \geq i$$

(interpreting i *as a binary string in* $\{0,1\}^n$ *in the natural way), and consequently* $|\gamma^{-1}(\{0,1\})| = i$.

Proof. Clearly the claim is satisfied before the first iteration of the while loop when $i = 0$ and PartialCandidates$^{(i)}$ = {AllUnknown}. Now, we must argue inductively. Suppose $1 \leq i \leq N$ and $\gamma' \in$ PartialCandidates$^{(i)}$. Then, it follows that there is some $\gamma \in$ PartialCandidates$^{(i-1)}$ and some $b \in \{0,1\}$ such that $\gamma' = \gamma_b$ where γ_b is as defined in the pseudocode. That is, γ_b is equal to γ except that the first ?-value (which occurs at $x^?_{\text{old}} = i - 1$ by the inductive hypothesis) is replaced by a b. Thus, we have

$$\gamma'(x) =? \iff \gamma(x) =? \wedge (x \neq x^?_{\text{old}}) \iff x > x^?_{\text{old}} \iff x \geq i$$

where the first equivalence comes from the definition of $\gamma_b = \gamma'$ and the second equivalence comes from the fact that $x^?_{\text{old}} = i - 1$. □

Next, we show that the PartialCandidates$^{(i)}$ never contains "redundant" partial functions.

Claim 36 (Author Claim 36)**.** *Before and after each iteration of the while loop, it is true that if γ' and γ'' are distinct elements of* PartialCandidates$^{(i)}$*, then no total function agrees with both γ' and γ''.*

Proof. Before the first iteration of the while loop runs, $i = 0$ and PartialCandidates$^{(0)}$ only contains the single partial function AllUnknown, so the claim clearly holds. Now we must show that the claim holds inductively. Assume $1 \leq i \leq N$. For contradiction, suppose there was some total function q that agrees with distinct elements μ and μ' from PartialCandidates$^{(i)}$. It follows that there exists some $b, b' \in \{0, 1\}$ and some (possibly not distinct) $\gamma, \gamma' \in$ PartialCandidates$^{(i-1)}$ such that $\mu = \gamma_b$ and $\mu' = \gamma'_{b'}$ (using the notation from the pseudocode where these functions γ_b and $\gamma'_{b'}$ are given by replacing the output of the first ?-valued input in γ or γ' respectively with a b-value or b'-value respectively). It follows that q must also agree with γ and γ'. Either $\gamma \neq \gamma'$ or not. If $\gamma \neq \gamma'$, then q agrees with two distinct elements from PartialCandidates$^{(i-1)}$ which contradicts the inductive hypothesis. Now suppose that $\gamma = \gamma'$. Then it must be that $b \neq b'$ (otherwise, $\mu = \mu'$ and we assumed they are distinct). But then, we have then γ and γ' have the same first ?-valued input $x^?$, so

$$b = \mu(x^?) = q(x^?) = \mu'(x^?) = b'$$

which contradicts that $b \neq b'$. □

Moreover, PartialCandidates$^{(i)}$ only contains partial functions that can be completed to total functions that pass a certain test.

Claim 37 (Author Claim 37)**.** *Before and after each iteration of the while loop, it is true that if $\gamma \in$* PartialCandidates$^{(i)}$ *then there exists a function g on n-inputs that agrees with γ such that*

$$L(\text{Select}[f, g]) \leq L(f) + n + 2.$$

Proof. Before the first iteration of the while loop runs, $i = 0$ and PartialCandidates$^{(0)}$ only contains one partial function (AllUnknown). The function f clearly agrees with AllUnknown, and it is easy to see that $L(\text{Select}[f, f]) = L(f) \leq L(f)+n+1$, as desired. Thus, the claim holds before the first iteration of the while loop. Moreover, the claim clearly continues holding inductively because before any γ_b is added to PartialCandidates$^{(i)}$, we check to see if the function g_{γ_b} satisfies $L(\text{Select}[f, g_{\gamma_b}]) \leq L(f) + n + 2$ and g_{γ_b} agrees with γ_b by construction. □

Finally, we show that PartialCandidates$^{(i)}$ always contains the partial functions we want.

Claim 38 (Author Claim 38)**.** *Suppose some function q is in an optimal subcomputation for f. Then before and after each iteration of the while loop there is a $\gamma \in$* PartialCandidates$^{(i)}$ *such that q agrees with γ. Moreover, once part 1 is finished, $q \in$* Candidates*.*

Proof. Fix some q as in the statement of the claim. Before the first iteration of the while loop runs, $i = 0$ and PartialCandidates$^{(0)}$ contains the all-? partial function AllUnknown, so q agrees with AllUnknown and the claim holds. Now, we must show the claim holds inductively. Assume $1 \leq i \leq N$. Then by

induction there exists a $\gamma \in \text{PartialCandidates}^{(i-1)}$ such that q agrees with γ. Let $b = q(i-1)$. Then q agrees with γ_b as defined in the pseudocode (replacing the first ?-value in γ with a b-value) since Claim 35 implies that

$$\gamma_b(x) = \begin{cases} b, & \text{if } x = i-1, \\ \gamma(x), & \text{otherwise.} \end{cases}$$

Thus, if we could show $\gamma_b \in \text{PartialCandidates}^{(i)}$, we would be done with showing the first part of the claim. From the pseudocode, it is clear $\gamma_b \in \text{PartialCandidates}^{(i)}$ if

$$L(\text{Select}[f, g_{\gamma_b}]) \leq L(f) + n + 2,$$

where g_{γ_b} is as defined in the code (the function given by replacing the ?-values in γ_b with ones) which we now prove. We already noted that $\gamma_b(x) = \begin{cases} b, & \text{if } x = i-1, \\ \gamma(x), & \text{otherwise.} \end{cases}$ Thus, appealing to Claim 35, we know that $\gamma_b(x) =? \iff x > x^?$ where $x^? \in \{0,1\}^n$ is the binary string equivalent to $i-1$ (note that $0 \leq i-1 \leq N-1$ so this makes sense). Hence, since q agrees with γ_b, we have that $g_{\gamma_b}(x) = q(x) \vee \text{GrtrThan}_{x^?}(x)$ where $\text{GrtrThan}_{x^?}(x) = 1$ if and only if $x > x^?$. Thus, we have that

$$\begin{aligned} \text{Select}[f, g_{\gamma_b}](x, z) &= \begin{cases} f(x), & \text{if } z = 0, \\ g_{\gamma_b}(x), & \text{if } z = 1 \end{cases} \\ &= \text{Select}[f, q](x, z) \vee (z \wedge \text{GrtrThan}_{x^?}(x)). \end{aligned}$$

Since $\{g, O, h\} \in \text{OptSubcomps}(f)$, we know that $L(\text{Select}[f, q]) = L(f) + 1$ by Lemma 23, and Proposition 15 implies that $L(\text{GrtrThan}_{x^?}) \leq n$. Hence, we have that $L(\text{Select}[f, g_{\gamma_b}]) \leq L(f) + n + 2$. Finally, we show that $q \in \text{Candidates}$ after part 1 finishes. Clearly, it suffices to show that $q \in \text{PartialCandidates}^{(N)}$ after part 1 finishes. We have already shown that there is a $\gamma \in \text{PartialCandidates}^{(N)}$ such that γ agrees with q. However, Claim 35 implies that γ is a total function and hence it equals q, so $q \in \text{PartialCandidates}^{(N)}$. □

Part 2: Finding a g, h pair within Candidates First, we note that any output by Algorithm 2 must be correct.

Claim 39 (Author Claim 39)**.** *Any value Algorithm 2 outputs must be an element of* $\text{OptSubcomps}(f)$*.*

Proof. Any output $\{g, O, h\}$ of Algorithm 2 must satisfy $f = gOh$ and $L(f) = L(g) + L(h)$ which implies $\{g, O, h\} \in \text{OptSubcomps}(f)$ by Lemma 21. □

Finally, we show that Algorithm 2 must output a value.

Claim 40 (Author Claim 40)**.** *Algorithm 2 must output a value (on input f).*

Proof. Since $L(f) \geq 2$, we have that $\text{OptSubcomps}(f)$ is non-empty. Let $\{g, O, h\} \in \text{OptSubcomps}(f)$. Claim 38 implies that $\{g, h\} \subseteq \text{Candidates}$. On the other hand, Lemma 21 implies that $L(f) = L(g) + L(h)$ and $f = gOh$. Thus, it is clear that part 2 will either output $\{g, O, h\}$ or output a value before that. □

5.2 Running Time of Algorithm 2

Fix some function f with n-inputs satisfying $L(f) \geq 2$. Let $N = 2^n$. We break the running time analysis into the two pieces of the algorithm.

Part 1 It is easy to see that the run time of part 1 can be bounded by

$$O\left(N + \sum_{i \in [N]} N \cdot |\,\text{PartialCandidates}^{(i)}\,|\right)$$

where $|\,\text{PartialCandidates}^{(i)}\,|$ indicates the size of $\text{PartialCandidates}^{(i)}$ after Algorithm 2 is finished adding elements to it. Moreover, we can bound the quantity $|\,\text{PartialCandidates}^{(i)}\,|$ as follows. Claim 37 implies that every partial function in $\text{PartialCandidates}^{(i)}$ must be consistent with some total function g on n-inputs satisfying $L(\text{Select}[f, g]) \leq L(f) + n + 2$. On the other hand, Claim 36 implies that any single (total) function can agree with at most partial function in $\text{PartialCandidates}^{(i)}$. Hence, we have that

$$|\,\text{PartialCandidates}^{(i)}\,| \leq |\{g : L(\text{Select}[f, g]) \leq L(f) + n + 2\}|$$

and Lemma 24 implies that

$$|\{g : L(\text{Select}[f, g]) \leq L(f)+n+2\}| \leq O(|\,\text{CanonOpt}^{n+1}\text{Formulas}(f)|\cdot N^2 \log N).$$

Thus, we have that part 1 runs in time at most $O(|\,\text{CanonOpt}^{n+1}\text{Formulas}(f)|\cdot N^4 \log N)$. Moreover, part 1 only makes oracle calls of length at most $2N$ (to calculate $L(\text{Select}[f, g_{\gamma_b}])$).

Part 2 It is easy to see that this part runs in time $O(N \cdot |\,\text{Candidates}\,|^2)$. Hence, since $\text{Candidates} = \text{PartialCandidates}^{(N)}$, the analysis in part 1 above implies that

$$|\,\text{Candidates}\,| \leq O(|\,\text{CanonOpt}^{n+1}\text{Formulas}(f)|\cdot N^2 \log N).$$

Thus, part 2 runs in time at most

$$O(|\,\text{CanonOpt}^{n+1}\text{Formulas}(f)|^2 \cdot N^5 \log^2 N).$$

Moreover, part 2 only makes oracle calls of length N.

In total Putting it all together, we have that Algorithm 2 runs in time at most

$$O(|\,\text{CanonOpt}^{n+1}\text{Formulas}(f)|^2 \cdot N^5 \log^2 N)$$

and only makes oracle queries of length $2N$. □

6 A worst-case randomized reduction

We now present a worst-case search to decision reduction for MFSP.

Theorem 41 (Author Theorem 41)**.** *There is a randomized algorithm solving Search-MFSP on inputs of length N in time $O(2^{.67N})$ given access to an oracle that solves MFSP on instances of length $2N$.*

Proof. We prove this theorem by giving an oracle algorithm solving Decomp-Problem and appealing to Theorem 20. We provide the pseudocode of our algorithm in Algorithm 3, which we recommend the reader look at before proceeding.

Algorithm 3. A randomized worst-case search to decision reduction for MFSP.

procedure WorstCaseOptimalSubcomputation(f)

Given the length-N truth table of a function f that takes n-inputs with $L(f) \geq 2$, this procedure returns an element $\{g, O, h\}$ of OptSubcomps(f). Set $s = \frac{2}{3} \cdot \frac{2^n}{\log n}$. Set $t = 2^{2N/3}$.

Part 1: Try random formulas For $i = 1, \ldots, t$ do:

1. Let G_i be a uniformly random binary tree with $L(f)$-leaves. (Section 6.2 discusses how to sample G_i.)
2. Turn G_i into a uniformly random formula ϕ_i by picking uniformly random gates from $\{\wedge, \vee\}$ and uniformly random input leaves from $\{0, 1, x_1, \ldots, x_n, \neg x_1, \ldots, \neg x_n\}$.
3. If ϕ_i computes f then: Write $\phi_i = \phi_{i,1} O \phi_{i,2}$. Let g and h be the function computed by $\phi_{i,1}$ and $\phi_{i,2}$ respectively. If $L(f) = L(g) + L(h)$ then return $\{g, O, h\}$. End if. End if.

End for.

Part 2: Generate a small list of candidates for g Set SmallFuncs = $\{g : g$ is a Boolean function with n-inputs and $L(g) \leq s\}$. Set Candidates = $\{g \in \text{SmallFuncs} : L(\text{Select}[f, g]) \leq L(f) + 1\}$.

Part 3: Try to find a g, h pair within Candidates For each pair of functions $(g, h) \in$ Candidates and for each gate $O \in \{\wedge, \vee\}$ do: If $f = gOh$ and $L(f) = L(g) + L(h)$ then return $\{g, O, h\}$. End if. End for.

Part 4: Try to find a g, h pair by looking at functions h satisfying $f = gOh$ Set SmallCandidates = $\{g \in$ Candidates : $L(g) \leq L(f) - s\}$. For each function $g \in$ SmallCandidates and for each $O \in \{\wedge, \vee\}$ do:

1. If $\forall x \in \{0,1\}^n \ \exists b \in \{0,1\}$ such that $g(x)Ob = f(x)$ then:
   (a) Let $h_{?,g} : \{0,1\}^n \to \{0,1,?\}$ be the unique partial function on n-inputs such that $\forall h$, $f = gOh \iff h$ agrees with $h_{?,g}$.

(b) For each total function h that agrees with $h_{?,g}$ do: If $f = gOh$ and $L(f) = L(g) + L(h)$ then return $\{g, O, h\}$. End if. End for.

End if.

End for.

end procedure

6.1 Correctness of Algorithm 3

In this section, we prove that Algorithm 3 has the desired input/output behavior. In our analysis, we will use s and t as parameters which we will set to the optimal values (which are written in the pseudocode) in Section 6.2 where we do the running time analysis for Algorithm 3. Fix some function f on n-inputs with $L(f) \geq 2$. We analyze the algorithm in parts.

Part 1 Since ϕ_i is chosen to have $L(f)$ leaves and the algorithm in part 1 checks if ϕ_i computes f before returning any value, the following claim is clear.

Claim 42 (Author Claim 42)**.** *Any output by Algorithm 3 returned in part 1 must be an element of* $\text{OptSubcomps}(f)$*.*

Moreover, we can lower bound the probability that Algorithm 3 returns a value in part 1 as follows. Recall that $\text{CanonOpt}^0\text{Formulas}(f)$ is the set of optimal canonical formulas for f. We will show that part 1 succeeds if this set is large.

Claim 43 (Author Claim 43)**.** *If* $t \geq 5 \cdot 2^{N(1+o(1))}/|\,\text{CanonOpt}^0\text{Formulas}(f)|$*, then part 1 of Algorithm 3 will return a value at least 99% of time.*

Proof. Since we are picking each s-leaf formula ϕ_i uniformly at random, the probability that any fixed formula computes f is at least

$$\frac{|\,\text{CanonOpt}^0\text{Formulas}(f)|}{\text{the total number of formulas with } L(f) \text{ leaves}}.$$

Combining Theorem 13 with Proposition 12 upper bounds the denominator by $2^{N(1+o(1))}$, so

$$\Pr[\phi_i \text{ computes } f] \geq \frac{|\,\text{CanonOpt}^0\text{Formulas}(f)|}{2^{N(1+o(1))}}.$$

Since each of these ϕ_i are chosen independently, we have that

$$\begin{aligned}\Pr[\exists i \in [t] \text{ such that } \phi_i \text{ computes } f] &\geq 1 - \left(1 - \frac{|\,\text{CanonOpt}^0\text{Formulas}(f)|}{2^{N(1+o(1))}}\right)^t \\ &\geq 1 - e^{-t\cdot|\,\text{CanonOpt}^0\text{Formulas}(f)|/2^{N(1+o(1))}} \\ &\geq 1 - e^{-5} \geq .99.\end{aligned}$$

Hence, with probability at least 99%, part 1 will find a ϕ_i computing f at which point it will clearly return a value. □

Part 2 In part 2, Algorithm 3 constructs the Candidates set. We prove two claims about this set. First, that it contains the functions we care about, and second that its size can be bounded using the size of the $\text{CanonOpt}^0\text{Formulas}(f)$ set.

Claim 44 (Author Claim 44)**.** *Suppose g is in an optimal subcomputation for f. Then $L(g) \leq s \implies g \in$ Candidates.*

Proof. Since $L(g) \leq s$, we know that $g \in$ SmallFuncs. Next, since g is in an optimal subcomputation for f, we have by Lemma 23 that $L(\text{Select}[f, g]) \leq L(f) + 1$, so g is an element of Candidates. □

Claim 45 (Author Claim 45)**.** $|\,\text{Candidates}\,| = O(|\,\text{CanonOpt}^0\text{Formulas}(f)| \cdot N \log N)$.

Proof. By construction, we have that Candidates $\subseteq \{g : L(\text{Select}[f, g]) = L(f)+1\}$. On the other hand, Lemma 24, we have that

$$|\{g : L(\text{Select}[f, g]) = L(f) + 1\}| \leq O(|\,\text{CanonOpt}^0\text{Formulas}(f)| \cdot N \log N).$$

□

Part 3 In Part 3, Algorithm 3 tries to find a g, h pair by looking within the Candidates set. We show this works as long as there is a $\{g, O, h\} \in \text{OptSubcomps}(f)$ where $L(g)$ and $L(h)$ are small. First, we note that part 3 can only return correct answers.

Claim 46 (Author Claim 46)**.** *Any output returned by Algorithm 3 in part 3 will be an element of* $\text{OptSubcomps}(f)$.

Proof. In order for a $\{g, O, h\}$ value to be returned in part 3 it must satisfy $f = gOh$ and $L(f) = L(g) + L(h)$. Thus, by Lemma 21, we know $\{g, O, h\} \in \text{OptSubcomps}(f)$. □

Next, we give sufficient conditions on which part 3 will return an answer.

Claim 47 (Author Claim 47)**.** *If there exists an element $\{g', O, h'\}$ of* $\text{OptSubcomps}(f)$ *such that $\max\{L(g'), L(h')\} \leq s$, then Algorithm 3 will return a value in part 3 or before.*

Proof. Suppose there exists an element $\{g', O, h'\}$ of $\text{OptSubcomps}(f)$ such that $\max\{L(g'), L(h')\} \leq s$, and assume that this procedure has not returned a value before it reaches part 3. Then by Claim 44, we have that both g' and h' are in Candidates. Moreover, since $\{g', O, h'\} \in \text{OptSubcomps}(f)$, we know that $f = g'O'h'$ and $L(f) = L(g) + L(h)$ by Lemma 21. Hence it is clear there are value the for loop will return an output if it reaches $g = g'$, $h = h'$, and $O = O'$ (although it could return a value before that). □

Part 4 In the final part of Algorithm 3, we look for matching h functions for g candidates with small complexity. First, we note that any output returned by part 4 must be correct by essentially the same proof as Claim 46 in part 3.

Claim 48 (Author Claim 48)**.** *Any output returned by Algorithm 3 in part 4 will be an element of* OptSubcomps(f)*.*

Next, we show sufficient conditions for part 4 returning an answer.

Claim 49 (Author Claim 49)**.** *If $s \geq L(f)/2$ and there exists a $\{g', O, h'\} \in$ OptSubcomps(f) such that $L(h') \geq s$, then Algorithm 3 will return a value in part 4 or earlier.*

Proof. Using Lemma 21, we have $L(f) = L(g') + L(h')$. Thus, since $s \geq L(f)/2$ and $L(h') \geq s$, we have that

$$L(g') = L(f) - L(h') \leq L(f) - s \leq 2s - s = s.$$

Hence, by Claim 44, we have that $g' \in$ Candidates. Moreover, since $L(g') \leq L(f) - s$, it follows that $g' \in$ SmallCandidates. Thus, it is clear that part 4 will return a value if its for loop ever reaches $g = g'$, $O = O'$, and $h = h'$ (though it could return a value before that). □

In total Finally, we can prove the correctness of the input/output behavior of Algorithm 3.

Claim 50 (Author Claim 50)**.** *If $s \geq L(f)/2$, then Algorithm 3 (run on input f) returns an element of* OptSubcomps(f)*.*

Proof. Put together, Claim 42, Claim 46, and Claim 48 ensures that any output returned by Algorithm 3 must be an element of OptSubcomps(f). Hence, it suffices to show that Algorithm 3 will always output a value. We divide into two cases. Either there exists an element $\{g', O, h'\} \in$ OptSubcomps(f) such that $\max\{L(g'), L(h')\} \leq s$ or not. If there exists such an element, then Claim 47 ensures that Algorithm 3 will output a value. Now suppose that for all $\{g', O, h'\} \in$ OptSubcomps(f) we have $\max\{L(g'), L(h')\} \geq s$. Since $L(f) \geq 2$, we know OptSubcomps(f) is non-empty by Proposition 18. Hence we can fix some $\{g', O, h'\} \in$ OptSubcomps(f) satisfying $\max\{L(g'), L(h')\} \geq s$. Without loss of generality we can assume that $L(g') \leq L(h')$. Thus, we have that $L(h') \geq s$, and by hypothesis $s \geq L(f)/2$, so Claim 49 ensures Algorithm 3 outputs a value. □

Thus Algorithm 3 is correct as long as $s \geq L(f)/2$. Indeed, in the next section we will set s so that $s \geq \max\{L(f)/2 : f \text{ takes } n\text{-inputs}\}$.

6.2 Runtime of Algorithm 3

In this subsection, we bound the runtime of Algorithm 3 and set s and t to the optimal values. We analyze Algorithm 3 in its parts.

Part 1 The for loop in part 1 clearly runs t times, so we just need to bound the running time of each iteration. Generating a uniformly random binary with $L(f)$-leaves can be done in linear time (see [15] for a survey of various approaches). The other operations in the for loop can clearly be done in time $O(N+L(f))$. Hence, all of part 1 runs in time $O(t\cdot(N+L(f)))$ which is $O(t\cdot N)$ using the worst-case formula upper bound from Theorem 13. Moreover, part 1 only makes oracle calls of length N (to calculate $L(f)$).

Part 2 Building the SmallFuncs set requires iterating through all formulas of size s (which is bounded by $2^{(1+o(1))\cdot s\log n}$ using Proposition 12) and then computing the truth table of each of these size s formulas (which can be done in time $O(Ns)$). Hence, computing SmallFuncs can be done in $O(N\cdot 2^{(1+o(1))\cdot s\log n})$ time. Moreover, $|\,\text{SmallFuncs}\,|$ is clearly upper bounded by the upper bound on the number of formulas of size s: $2^{(1+o(1))\cdot s\log n}$. Next, building the Candidates set can be done in time $O(|\,\text{SmallFuncs}\,|+N)=O(N\cdot 2^{(1+o(1))\cdot s\log n})$, and we use oracle calls of length $2N$ in this step (for Select$[f,g]$). Hence, part 2 runs in time $O(N\cdot 2^{(1+o(1))\cdot s\log n})$. We will make use of the following claim later, which bounds the size of the Candidates set if part 1 did not return a value.

Claim 51 (Author Claim 51)**.** *Fix some function f. Then with 99% probability (over the algorithm's choice of random formulas) either Algorithm 3 on input f returns before reaching part two or*

$$|\,\text{Candidates}\,|\leq\frac{2^{N(1+o(1))}}{t}.$$

Proof. Suppose that Algorithm 3 reaches part two on input f and $|\,\text{Candidates}\,|>2^{N(1+o(1))}/t$. Then Claim 45 implies that

$$\frac{2^{N(1+o(1))}}{t}=O(|\,\text{CanonOpt}^0\text{Formulas}(f)|N\log n)$$

which implies that

$$|\,\text{CanonOpt}^0\text{Formulas}(f)|\geq\frac{2^{N(1+o(1))}}{t}.$$

Hence, Claim 43 implies that Algorithm 3 will return in part 1 with 99% probability. □

Part 3 It is easy to see that part 3 runs in time $O(|\,\text{Candidates}\,|^2+N)$ and makes oracle calls of length N. Thus, using Claim 51, we have that with 99% probability part 3 runs in time $2^{2N(1+o(1))}/t^2$.

Part 4 Computing SmallCandidates can be done in $O(|\,\text{Candidates}\,|+N)$ time, and the outer for loop runs at most $O(|\,\text{Candidates}\,|)$ many times. It remains to bound the running time of each iteration of the outer for loop. The if condition can be checked in $O(N)$ time. Constructing $h_{?,g}$ also takes $O(N)$ time (similar to the if condition, just iterate through each input $x\in\{0,1\}^n$ and see which values of $b\in\{0,1\}$ satisfy $f(x)=g(x)Ob$). Each iteration of the

inner for loop takes $O(N)$ time. Finally, the inner for loop runs $2^{|h_{?,g}^{-1}(?)|}$ many times. Thus, the total running time for part 4 is

$$O\left(|\text{ Candidates }| \cdot \left(N + N \cdot 2^{\max_g\{|h_{?,g}^{-1}(?)|\}}\right)\right)$$

time. Moreover, we can bound the quantity $\max_g\{|h_{?,g}^{-1}(?)|\}$ as follows.

Claim 52 (Author Claim 52)**.** $\max_g\{|h_{?,g}^{-1}(?)|\} \leq (1 + o(1))(N - s \log n)$.

Proof. Since any function h that agrees with $h_{?,g}$ satisfies $gOh = f$ and, we have that

$$L(f) \leq L(g) + L(h_{?,g}).$$

Since $L(g) \leq L(f) - s$ (since $g \in$ SmallCandidates), we have that $L(h_{?,g}) \geq s$. On the other hand, the upper bound on the formula complexity of partial functions from Theorem 16 implies that

$$L(h_{?,g}) \leq (1 + o(1)) \left(1 - \frac{|h_{?,g}^{-1}(?)|}{N}\right) \frac{N}{\log n}.$$

Hence

$$s \leq (1 + o(1)) \left(1 - \frac{|h_{?,g}^{-1}(?)|}{N}\right) \frac{N}{\log n}$$

so

$$|h_{?,g}^{-1}(?)| \leq N - \frac{s \log n}{1 + o(1)} \leq (1 + o(1))(N - s \log n).$$

□

Thus, using Claim 51, we have that with 99% probability part 4 runs in $2^{2N(1+o(1))}/t^2$.

$$O\left(\frac{2^{N(1+o(1))}}{t} \cdot N \cdot 2^{(1+o(1))(N - s \log n)}\right) = \frac{2^{(1+o(1))(2N - s \log n)}}{t}$$

time.

In total Thus, we get that with 99% probability Algorithm 3 runs in time

$$O(t \cdot N) + O(N \cdot 2^{(1+o(1)) \cdot s \log n}) + \frac{2^{2N(1+o(1))}}{t^2} + \frac{2^{(1+o(1))(2N - s \log n)}}{t}.$$

Letting $s = \frac{2}{3} \frac{2^n}{\log n}$ and $t = 2^{(2/3)N}$, we get that the running time is bounded by

$$2^{(1+o(1))(2/3)N}.$$

Moreover, s will satisfy $s \geq L(f)/2$ (as required for the correctness of the algorithm) for all f with n-inputs when n is sufficiently large by Theorem 13. □

References

[1] Eric Allender. The new complexity landscape around circuit minimization. In Alberto Leporati, Carlos Martín-Vide, Dana Shapira, and Claudio Zandron, editors, Language and Automata Theory and Applications - 14th International Conference, LATA 2020, Milan, Italy, March 4-6, 2020, Proceedings, volume 12038 of Lecture Notes in Computer Science, pages 3–16. Springer, 2020. doi:10.1007/978-3-030-40608-0_1.

[2] Eric Allender, Harry Buhrman, Michal Koucký, Dieter van Melkebeek, and Detlef Ronneburger. Power from random strings. SIAM J. Comput., 35(6):1467–1493, 2006.

[3] Eric Allender and Bireswar Das. Zero knowledge and circuit minimization. Inf. Comput., 256:2–8, 2017.

[4] Eric Allender, Michal Koucký, Detlef Ronneburger, and Sambuddha Roy. The pervasive reach of resource-bounded kolmogorov complexity in computational complexity theory. Journal of Computer and System Sciences, 77(1):14–40, 2011.

[5] David Buchfuhrer and Christopher Umans. The complexity of boolean formula minimization. J. Comput. Syst. Sci., 77(1):142–153, 2011.

[6] Marco L. Carmosino, Russell Impagliazzo, Valentine Kabanets, and Antonina Kolokolova. Learning algorithms from natural proofs. In Proceedings of the 31st Conference on Computa- tional Complexity, 2016.

[7] Alexander Golovnev, Rahul Ilango, Russell Impagliazzo, Valentine Kabanets, Antonina Koloko- lova, and Avishay Tal. Ac0 [p] lower bounds against MCSP via the coin problem. In ICALP, volume 132 of LIPICS, pages 66:1–66:15, 2019.

[8] Shuichi Hirahara. Non-black-box worst-case to average-case reductions within NP. In 59th IEEE Annual Symposium on Foundations of Computer Science, FOCS, pages 247–258, 2018.

[9] Shuichi Hirahara, Igor Carboni Oliveira, and Rahul Santhanam. NP-hardness of minimum circuit size problem for OR-AND-MOD circuits. In 33rd Computational Complexity Conference, CCC, volume 102, pages 5:1–5:31, 2018.

[10] Valentine Kabanets and Jin-Yi Cai. Circuit minimization problem. In Proceedings of the Thirty-Second Annual ACM Symposium on Theory of Computing, STOC '00, page 73–79, 2000.

[11] S. A. Lozhkin. Tighter bounds on the complexity of control systems from some classes. Mat. Voprosy Kibernetiki 6, pages 189–214 (in Russian), 1996.

[12] Oleg B. Lupanov. Complexity of formula realization of functions of logical algebra. Problemy Kibernetiki, 3:61–80, 1960.

[13] William J. Masek. Some NP-complete set covering problems. Unpublished Manuscript, 1979.

[14] Cody D. Murray and R. Ryan Williams. On the (non) np-hardness of computing circuit complexity. Theory of Computing, 13(1):1–22, 2017.

[15] Erkki Mäkinen. Generating random binary trees — a survey. Information Sciences, 115(1):123– 136, 1999.

[16] Nicholas Pippenger. Information theory and the complexity of boolean functions. Mathematical Systems Theory, 10:129–167, January 1977.

[17] Alexander A. Razborov and Steven Rudich. Natural proofs. J. Comput. Syst. Sci., 55(1):24–35, 1997.

[18] Michael Rudow. Discrete logarithm and minimum circuit size. Inf. Process. Lett., 128:1–4, 2017.

[19] Rahul Santhanam. Pseudorandomness and the minimum circuit size problem. In Thomas Vidick, editor, 11th Innovations in Theoretical Computer Science Conference, ITCS, volume 151 of LIPIcs, pages 68:1–68:26, 2020.

[20] B. A. Trakhtenbrot. A survey of russian approaches to perebor (brute-force searches) algorithms. IEEE Ann. Hist. Comput., 6(4):384–400, October 1984.